# ML Architecture

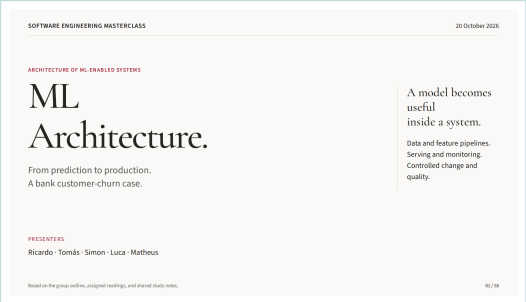
Speaker Notes

A bank customer-churn case — 20 October 2026

Each page shows one slide exactly as presenters will see it, the key points to cover, delivery cues, and a worked spoken-example paragraph to practice from. The example is a model to rehearse and adapt, not a transcript to read verbatim. Animated builds are marked; press the arrow key the stated number of times before moving to the next slide.

## 01 / 55 ML Architecture

Block 1 / Foundations • All presenters




**CUES**

- New block: **Block 1 / Foundations**.
- Presenter: **All presenters**.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

**TO SAY**

- From a trained model to a maintainable, observable, governed system.
- **Software engineering** for systems that depend on learned behavior.
- **One shared case**: bank customer-churn prediction.
- **Two blocks**: model-to-system foundations, then architecture and quality.
- **Five presenters**: Ricardo, Tomás, Simon, Luca, and Matheus.
- Bank churn supplies the shared case. Examples, records, schedules, and incidents are illustrative.
- *A useful model needs an engineered system around it.*

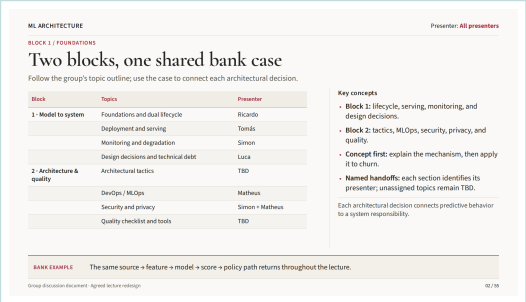
**SPOKEN EXAMPLE — practice this out loud**

*Today we're looking at what it takes to turn a trained model into a system you can actually trust and run in production. We'll use one running example throughout: a bank trying to predict which customers might leave, or "churn." You'll see that building the model is really just the first step — the real engineering work is everything that happens around it: serving predictions reliably, watching for problems, and keeping the whole system accountable. Five of us will walk through this together, in two parts: first the foundations, then architecture and quality.*

Reading anchor: Bass, Ch. 14 · Kalinowski, Ch. 13–14

## 02 / 55 Two blocks, one shared bank case

Block 1 / Foundations • All presenters




### CUES

- Presenter: **All presenters**.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Follow the group's topic outline; use the case to connect each architectural decision.
- **Block 1**: lifecycle, serving, monitoring, and design decisions.
- **Block 2**: tactics, MLOps, security, privacy, and quality.
- **Concept first**: explain the mechanism, then apply it to churn.
- **Named handoffs**: each section identifies its presenter; unassigned topics remain TBD.
- *At our bank,* the same source → feature → model → score → policy path returns throughout the lecture.
- *Each architectural decision connects predictive behavior to a system responsibility.*

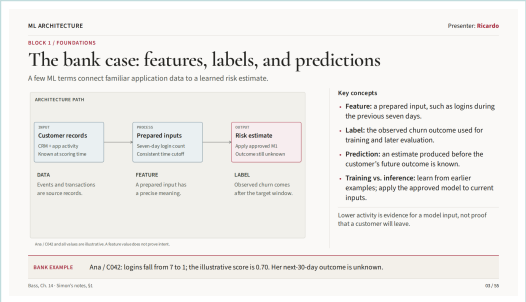
### SPOKEN EXAMPLE — practice this out loud

*We've split today's talk into two blocks. The first covers the lifecycle of a model — how it's served, monitored, and designed. The second moves into tactics, operations, security, and quality. Rather than jumping between abstract ideas, we'll always ground each concept in the same story: a bank scoring customers for churn risk. So whenever we introduce a new idea, ask yourself: what does this actually mean for that one bank and that one model?*

Reading anchor: Group discussion document · Agreed lecture redesign

## 03 / 55 The bank case: features, labels, and predictions

Block 1 / Foundations • Ricardo




### CUES

- Presenter: **Ricardo**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- A few ML terms connect familiar application data to a learned risk estimate.
- **Feature**: a prepared input, such as logins during the previous seven days.
- **Label**: the observed churn outcome used for training and later evaluation.
- **Prediction**: an estimate produced before the customer's future outcome is known.
- **Training vs. inference**: learn from earlier examples; apply the approved model to current inputs.
- Ana / C042: logins fall from 7 to 1; the illustrative score is 0.70. Her next-30-day outcome is unknown.
- *Lower activity is evidence for a model input, not proof that a customer will leave.*

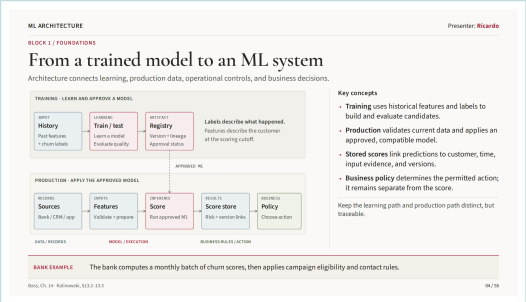
### SPOKEN EXAMPLE — practice this out loud

*Let's meet the example that will follow us through the whole talk. Ana is a fictional customer, customer number C042. Over the last week, her app logins dropped from seven down to just one — that's a feature, a piece of prepared input data. Based on that and other signals, our model estimates her churn risk at 0.70. That's a prediction — a guess made before we know what actually happens. Only later, once we observe whether she actually leaves, do we get the label. It's really important to say clearly: a lower login count is evidence, not proof. Ana hasn't left the bank — not yet, maybe not ever.*

Reading anchor: Bass, Ch. 14 · Simon's notes, §1

## 04 / 55 From a trained model to an ML system

Block 1 / Foundations • Ricardo




### CUES

- Presenter: **Ricardo**.
- Animated build: click 4 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Architecture connects learning, production data, operational controls, and business decisions.
- **Training** uses historical features and labels to build and evaluate candidates.
- **Production** validates current data and applies an approved, compatible model.
- **Stored scores** link predictions to customer, time, input evidence, and versions.
- **Business policy** determines the permitted action; it remains separate from the score.
- *At our bank,* the bank computes a monthly batch of churn scores, then applies campaign eligibility and contact rules.
- *Keep the learning path and production path distinct, but traceable.*

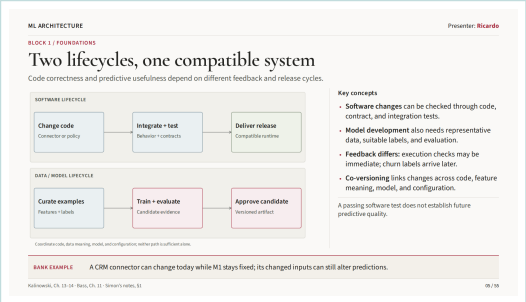
### SPOKEN EXAMPLE — practice this out loud

*So how does a model like the one that scored Ana actually become part of a real system? There's a training side, where we learn from historical data, and a production side, where we apply an already-approved model to current customers. Every score we produce gets stored together with who it was for, when it happened, and which model version made it. And crucially, the business policy — who actually gets contacted — sits completely separate from that score. At our bank, that means the churn score is computed once a month, in a batch, and only afterward does the campaign team apply its own eligibility rules.*

Reading anchor: Bass, Ch. 14 · Kalinowski, §13.2–13.3

## 05 / 55 Two lifecycles, one compatible system

Block 1 / Foundations • Ricardo




### CUES

- Presenter: **Ricardo**.
- Animated build: click 3 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Code correctness and predictive usefulness depend on different feedback and release cycles.
- **Software changes** can be checked through code, contract, and integration tests.
- **Model development** also needs representative data, suitable labels, and evaluation.
- **Feedback differs**: execution checks may be immediate; churn labels arrive later.
- **Co-versioning** links changes across code, feature meaning, model, and configuration.
- *At our bank,* a CRM connector can change today while M1 stays fixed; its changed inputs can still alter predictions.
- *A passing software test does not establish future predictive quality.*

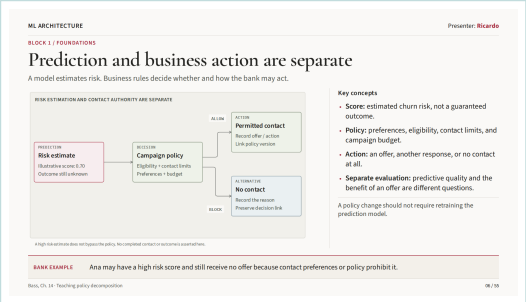
### SPOKEN EXAMPLE — practice this out loud

*Here's something that trips a lot of teams up: code and models don't change on the same clock. You can test a code change quickly — does it compile, does it pass its tests. But testing whether a model is still predictively useful takes much longer, because you need real labels, and those arrive late. Imagine our bank's CRM connector changes today. The code deploys fine, the tests pass — but the model, M1, hasn't changed at all. Yet its inputs have, and that alone can shift what it predicts. So a green test suite is not a guarantee that your predictions are still good.*

Reading anchor: Kalinowski, Ch. 13–14 · Bass, Ch. 11 · Simon's notes, §1

## 06 / 55 Prediction and business action are separate

Block 1 / Foundations • Ricardo




### CUES

- Presenter: **Ricardo**.
- Animated build: click 3 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- A model estimates risk. Business rules decide whether and how the bank may act.
- **Score**: estimated churn risk, not a guaranteed outcome.
- **Policy**: preferences, eligibility, contact limits, and campaign budget.
- **Action**: an offer, another response, or no contact at all.
- **Separate evaluation**: predictive quality and the benefit of an offer are different questions.
- Ana may have a high risk score and still receive no offer because contact preferences or policy prohibit it.
- *A policy change should not require retraining the prediction model.*

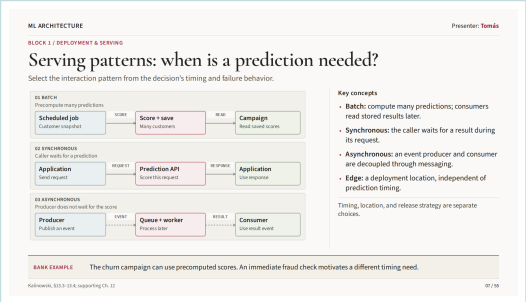
### SPOKEN EXAMPLE — practice this out loud

*A model's score is just an estimate — never a guaranteed outcome. What happens next is governed by business policy: contact preferences, who's eligible, how many times someone can be contacted, and the campaign's budget. So picture Ana again. Even with a high churn-risk score, she might receive no offer at all, simply because policy says no. That separation matters a lot in practice — it means if the bank wants to change its contact rules tomorrow, nobody needs to retrain the model to do it.*

Reading anchor: Bass, Ch. 14 · Teaching policy decomposition

## 07 / 55 Serving patterns: when is a prediction needed?

Block 1 / Deployment & serving • Tomás




**CUES**

- New block: **Block 1 / Deployment & serving**.
- Presenter: **Tomás**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

**TO SAY**

- Select the interaction pattern from the decision's timing and failure behavior.
- **Batch**: compute many predictions; consumers read stored results later.
- **Synchronous**: the caller waits for a result during its request.
- **Asynchronous**: an event producer and consumer are decoupled through messaging.
- **Edge**: a deployment location, independent of prediction timing.
- *At our bank,* the churn campaign can use precomputed scores. An immediate fraud check motivates a different timing need.
- *Timing, location, and release strategy are separate choices.*

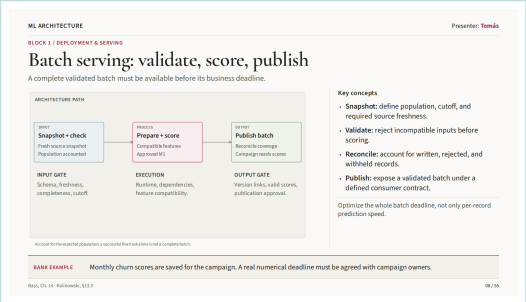
**SPOKEN EXAMPLE — practice this out loud**

*Now Tomás is going to walk us through how predictions actually get delivered. The first question to ask is timing: does someone need this answer right now, or can it wait? For our churn case, a batch works fine — scores get computed overnight and read later by the campaign. But imagine a fraud check instead: that needs an answer synchronously, while the customer is still mid-transaction. And there's a third option, asynchronous, where producer and consumer are decoupled through messaging. Notice that where the model physically runs — centrally or at the edge — is a completely separate decision from any of this.*

Reading anchor: Kalinowski, §13.3–13.4; supporting Ch. 12

## 08 / 55 Batch serving: validate, score, publish

Block 1 / Deployment & serving • Tomás




### CUES

- Presenter: **Tomás**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- A complete validated batch must be available before its business deadline.
- **Snapshot**: define population, cutoff, and required source freshness.
- **Validate**: reject incompatible inputs before scoring.
- **Reconcile**: account for written, rejected, and withheld records.
- **Publish**: expose a validated batch under a defined consumer contract.
- *At our bank,* monthly churn scores are saved for the campaign. A real numerical deadline must be agreed with campaign owners.
- *Optimize the whole batch deadline, not only per-record prediction speed.*

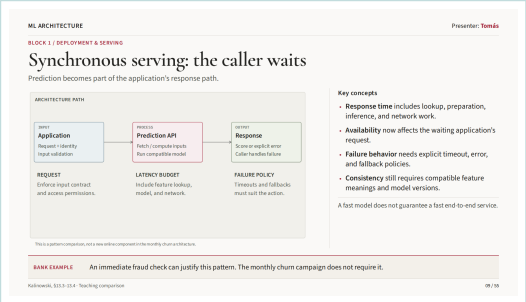
### SPOKEN EXAMPLE — practice this out loud

*Batch serving means computing lots of predictions at once, ahead of when they're needed. For the bank, that means defining a clear snapshot: which customers, as of when, using which sources. Before scoring anyone, we validate the inputs and reject anything that doesn't meet the contract. We reconcile what got written, what got rejected, and what had to be withheld. Only then do we publish — expose a complete, validated batch under a contract the campaign team can rely on. The real goal isn't making any single prediction fast; it's hitting the deadline for the whole batch.*

Reading anchor: Bass, Ch. 14 · Kalinowski, §13.3

## 09 / 55 Synchronous serving: the caller waits

Block 1 / Deployment & serving • Tomás




### CUES

- Presenter: **Tomás**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Prediction becomes part of the application’s response path.
- **Response time** includes lookup, preparation, inference, and network work.
- **Availability** now affects the waiting application’s request.
- **Failure behavior** needs explicit timeout, error, and fallback policies.
- **Consistency** still requires compatible feature meanings and model versions.
- *At our bank,* an immediate fraud check can justify this pattern. The monthly churn campaign does not require it.
- *A fast model does not guarantee a fast end-to-end service.*

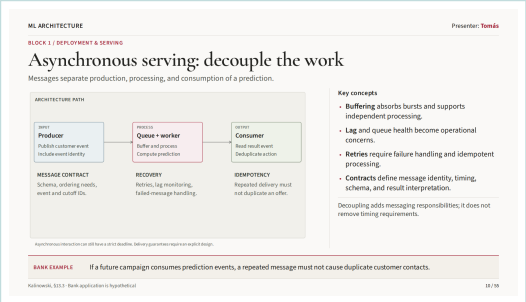
### SPOKEN EXAMPLE — practice this out loud

*Synchronous serving puts the model directly on the critical path of a user’s request — they’re waiting right now for an answer. That means your response time now has to include looking things up, preparing features, running inference, and the network round-trip. And since the caller is blocked, you need explicit policies for what happens on a timeout or an error. A real-time fraud check is a good candidate for this pattern. Our monthly churn campaign is not — there’s no customer sitting there waiting for that score.*

Reading anchor: Kalinowski, §13.3–13.4 · Teaching comparison

## 10 / 55 Asynchronous serving: decouple the work

Block 1 / Deployment & serving • Tomás




### CUES

- Presenter: **Tomás**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Messages separate production, processing, and consumption of a prediction.
- **Buffering** absorbs bursts and supports independent processing.
- **Lag** and queue health become operational concerns.
- **Retries** require failure handling and idempotent processing.
- **Contracts** define message identity, timing, schema, and result interpretation.
- *At our bank,* if a future campaign consumes prediction events, a repeated message must not cause duplicate customer contacts.
- *Decoupling adds messaging responsibilities; it does not remove timing requirements.*

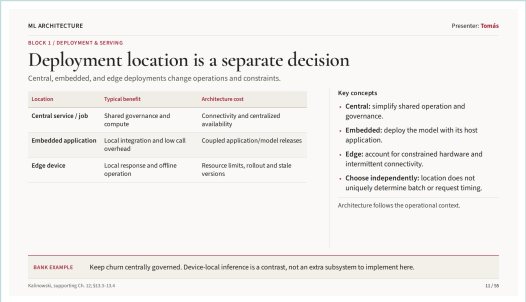
### SPOKEN EXAMPLE — practice this out loud

*Asynchronous serving uses messages to separate who produces a prediction from who eventually consumes it. That buffering is useful — it smooths out bursts of work. But it brings its own concerns: how far behind is the queue, and what happens if a message gets retried? If our bank ever built a campaign that reacted to prediction events in real time, a retried message must never cause the same customer to be contacted twice. Decoupling doesn't remove your timing obligations — it just moves them into a messaging contract.*

Reading anchor: Kalinowski, §13.3 · Bank application is hypothetical

## 11 / 55 Deployment location is a separate decision

Block 1 / Deployment & serving • Tomás




**CUES**

- Presenter: **Tomás**.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

**TO SAY**

- Central, embedded, and edge deployments change operations and constraints.
- **Central**: simplify shared operation and governance.
- **Embedded**: deploy the model with its host application.
- **Edge**: account for constrained hardware and intermittent connectivity.
- **Choose independently**: location does not uniquely determine batch or request timing.
- *At our bank,* keep churn centrally governed. Device-local inference is a contrast, not an extra subsystem to implement here.
- *Architecture follows the operational context.*

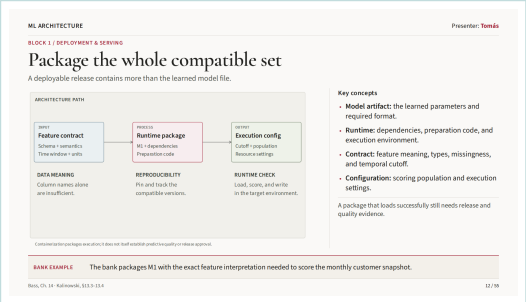
**SPOKEN EXAMPLE — practice this out loud**

*Where a model physically runs is its own decision, independent of everything else we've covered. Centrally hosted models are easier to govern as one shared system. Embedded models ship inside their host application. And edge deployments have to deal with limited hardware and patchy connectivity. For our churn model, we keep everything centrally governed — there's no reason to push it onto a customer's device. The point is: location, timing, and release strategy are three separate knobs, not one.*

Reading anchor: Kalinowski, supporting Ch. 12; §13.3–13.4

## 12 / 55 Package the whole compatible set

Block 1 / Deployment & serving • Tomás




### CUES

- Presenter: **Tomás**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- A deployable release contains more than the learned model file.
- **Model artifact**: the learned parameters and required format.
- **Runtime**: dependencies, preparation code, and execution environment.
- **Contract**: feature meaning, types, missingness, and temporal cutoff.
- **Configuration**: scoring population and execution settings.
- *At our bank,* the bank packages M1 with the exact feature interpretation needed to score the monthly customer snapshot.
- *A package that loads successfully still needs release and quality evidence.*

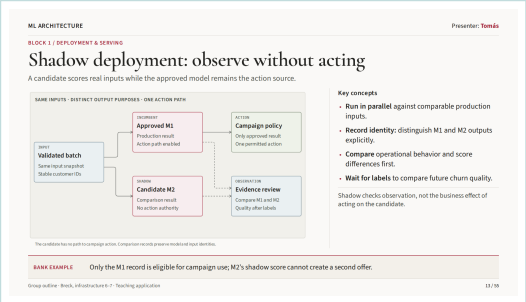
### SPOKEN EXAMPLE — practice this out loud

*When you deploy a model, the model file itself is really the smallest part of what you're shipping. You also need the runtime it depends on, the code that prepares its features, and a contract describing exactly what each feature means, including how missing values are handled. For the bank, that means M1 is packaged together with the precise definition of its features — not just the model weights. Even when a package installs and loads without errors, that still tells you nothing about whether it's actually ready for release.*

Reading anchor: Bass, Ch. 14 · Kalinowski, §13.3–13.4

## 13 / 55 Shadow deployment: observe without acting

Block 1 / Deployment & serving • Tomás




### CUES

- Presenter: **Tomás**.
- Animated build: click 4 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- A candidate scores real inputs while the approved model remains the action source.
- **Run in parallel** against comparable production inputs.
- **Record identity**: distinguish M1 and M2 outputs explicitly.
- **Compare** operational behavior and score differences first.
- **Wait for labels** to compare future churn quality.
- *At our bank,* only the M1 record is eligible for campaign use; M2's shadow score cannot create a second offer.
- *Shadow checks observation, not the business effect of acting on the candidate.*

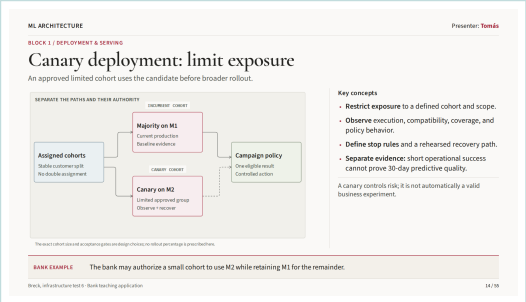
### SPOKEN EXAMPLE — practice this out loud

*Shadow deployment lets a new candidate model quietly score the same real inputs as the approved model, without ever influencing a real decision. We run both in parallel, and we're careful to record which output came from which model — M1 or the candidate, M2. At this stage we're only comparing operational behavior and how the scores differ; we still need to wait for actual outcomes to judge predictive quality. At our bank, only M1's record is ever eligible to trigger a campaign offer — M2 running in the shadows can never create a second, invisible offer.*

Reading anchor: Group outline · Breck, infrastructure 6–7 · Teaching application

## 14 / 55 Canary deployment: limit exposure

Block 1 / Deployment & serving • Tomás




### CUES

- Presenter: **Tomás**.
- Animated build: click 3 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- An approved limited cohort uses the candidate before broader rollout.
- **Restrict exposure** to a defined cohort and scope.
- **Observe** execution, compatibility, coverage, and policy behavior.
- **Define stop rules** and a rehearsed recovery path.
- **Separate evidence**: short operational success cannot prove 30-day predictive quality.
- *At our bank,* the bank may authorize a small cohort to use M2 while retaining M1 for the remainder.
- *A canary controls risk; it is not automatically a valid business experiment.*

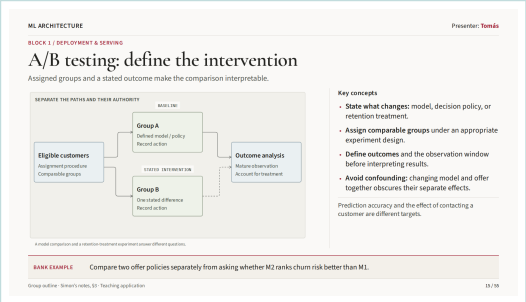
### SPOKEN EXAMPLE — practice this out loud

*A canary release lets a small, defined group of customers actually experience the new model, while everyone else stays on the one we trust. We watch execution, compatibility, and how it behaves under real policy — and critically, we define ahead of time what would make us stop, and how we'd roll back. Say the bank authorizes M2 for just a small cohort while keeping M1 everywhere else. One thing to be careful about: a canary running smoothly for a few days tells you the system didn't break. It does not, by itself, prove the new model is better over a full 30-day churn window.*

Reading anchor: Breck, infrastructure test 6 · Bank teaching application

## 15 / 55 A/B testing: define the intervention

Block 1 / Deployment & serving • Tomás




### CUES

- Presenter: **Tomás**.
- Animated build: click 3 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Assigned groups and a stated outcome make the comparison interpretable.
- **State what changes**: model, decision policy, or retention treatment.
- **Assign comparable groups** under an appropriate experiment design.
- **Define outcomes** and the observation window before interpreting results.
- **Avoid confounding**: changing model and offer together obscures their separate effects.
- *At our bank,* compare two offer policies separately from asking whether M2 ranks churn risk better than M1.
- *Prediction accuracy and the effect of contacting a customer are different targets.*

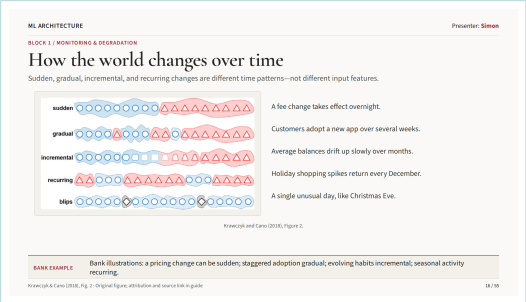
### SPOKEN EXAMPLE — practice this out loud

*A proper A/B test starts by naming exactly what's changing — is it the model, the decision policy, or the retention offer itself? Then you assign genuinely comparable groups, and you decide upfront what outcome you're measuring and over what window. The trap to avoid is confounding: if you change both the model and the offer at the same time, you'll never know which one caused the result. So we'd test two offer policies in one experiment, and separately test whether M2 actually ranks churn risk better than M1 — never both questions in the same test.*

Reading anchor: Group outline · Simon's notes, §3 · Teaching application

## 16 / 55 How the world changes over time

Block 1 / Monitoring & degradation • Simon




**CUES**

- New block: **Block 1 / Monitoring & degradation**.
- Presenter: **Simon**.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.
- Adaptation note on file for this slide; do not read aloud unless asked.

**TO SAY**

- Sudden, gradual, incremental, and recurring changes are different time patterns—not different input features.
- **Sudden**: the old regime switches abruptly.
- **Gradual**: old and new regimes coexist during a transition.
- **Incremental / recurring**: intermediate change differs from a returning regime.
- **Blips**: isolated noise is not necessarily a lasting regime change.
- Bank illustrations: a pricing change can be sudden; staggered adoption gradual; evolving habits incremental; seasonal activity recurring.
- *First ask how change unfolds over time. Then ask what changed: input data, outcome relationships, or data validity.*

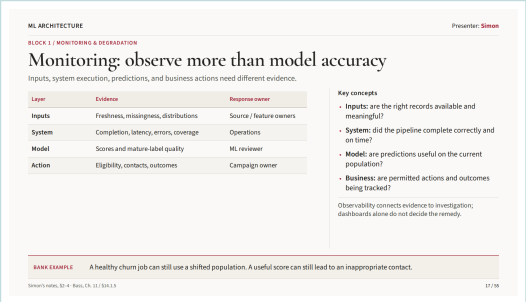
**SPOKEN EXAMPLE — practice this out loud**

*Before we talk about what can go wrong with a model over time, it helps to separate out how change happens, in time, from what actually changed. A sudden change flips overnight — think of a bank changing its fees overnight. A gradual change has the old and new situations overlapping for a while, like customers slowly adopting a new app. An incremental change drifts through in-between states, the way average balances creep up over months. A recurring change comes back on a schedule, like holiday shopping every December. And a blip is just one unusual day, like Christmas Eve — noise, not a lasting shift. Keep these patterns separate in your head from the next question, which is what actually changed: the data coming in, the relationship to the outcome, or just plain broken data.*

Reading anchor: Krawczyk & Cano (2018), Fig. 2 · Original figure; attribution and source link in guide

## 17 / 55 Monitoring: observe more than model accuracy

Block 1 / Monitoring & degradation • Simon




**CUES**

- Presenter: **Simon**.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

**TO SAY**

- Inputs, system execution, predictions, and business actions need different evidence.
- **Inputs**: are the right records available and meaningful?
- **System**: did the pipeline complete correctly and on time?
- **Model**: are predictions useful on the current population?
- **Business**: are permitted actions and outcomes being tracked?
- *At our bank,* a healthy churn job can still use a shifted population. A useful score can still lead to an inappropriate contact.
- *Observability connects evidence to investigation; dashboards alone do not decide the remedy.*

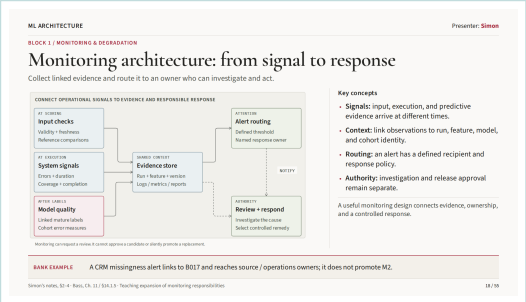
**SPOKEN EXAMPLE — practice this out loud**

*It's tempting to think monitoring a model just means watching its accuracy. It's really watching four different things. Are the inputs actually there and meaningful? Did the pipeline that produces them run correctly and on time? Are the predictions still useful for today's customers? And are the actions the business takes with those predictions actually being tracked? Here's the trap: a churn job can run perfectly, every record present and on time, and still be scoring a population that's shifted. And a perfectly good score can still lead to a contact that shouldn't have happened. A dashboard full of green checkmarks doesn't decide what to do next — it just tells you where to look.*

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5

## 18 / 55 Monitoring architecture: from signal to response

Block 1 / Monitoring & degradation • Simon




### CUES

- Presenter: **Simon**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Collect linked evidence and route it to an owner who can investigate and act.
- **Signals**: input, execution, and predictive evidence arrive at different times.
- **Context**: link observations to run, feature, model, and cohort identity.
- **Routing**: an alert has a defined recipient and response policy.
- **Authority**: investigation and release approval remain separate.
- *At our bank,* a CRM missingness alert links to B017 and reaches source / operations owners; it does not promote M2.
- *A useful monitoring design connects evidence, ownership, and a controlled response.*

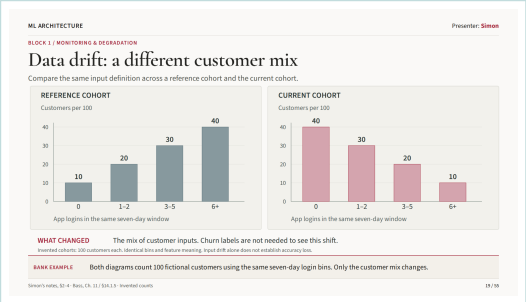
### SPOKEN EXAMPLE — practice this out loud

*Good monitoring isn't just collecting numbers — it's getting the right evidence to the right person. Different signals — about inputs, about execution, about predictions — show up on different timelines, so we need to link them back to a specific run, feature version, model, and customer cohort. And every alert needs somewhere to go: a defined recipient and a defined response. Picture a missing-data alert from the CRM migration — it links to the failed batch we'll call B017, and it goes to the data and operations owners. It does not, on its own, promote a new model. Investigating a problem and approving a release are always kept as separate authorities.*

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5 · Teaching expansion of monitoring responsibilities

## 19 / 55 Data drift: a different customer mix

Block 1 / Monitoring & degradation • Simon




**CUES**

- Presenter: **Simon**.
- Animated build: click 1 time to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

**TO SAY**

- Compare the same input definition across a reference cohort and the current cohort.
- **What changes**: the distribution of valid model inputs.
- **Bank example**: a campaign attracts more low-activity customers.
- **Available now**: compare inputs without waiting for churn labels.
- **Limit**: a changed distribution alone does not prove lost accuracy.
- *At our bank,* both diagrams count 100 fictional customers using the same seven-day login bins. Only the customer mix changes.
- *Choose a meaningful reference, comparable cohorts, and a defined response.*

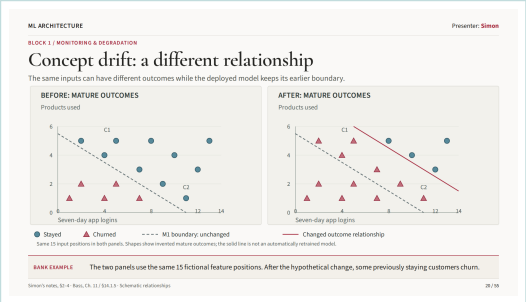
**SPOKEN EXAMPLE — practice this out loud**

*Data drift means the mix of customers coming in has changed, even though the feature definition stayed exactly the same. Say a marketing campaign suddenly attracts a lot more low-activity customers than usual — that's a shift we can actually see right away, just by comparing input distributions, without waiting weeks for churn labels to come in. In our illustration, both cohorts are 100 fictional customers using identical seven-day login bins — only the mix across those bins has moved. But here's the limit to keep in mind: seeing that shift alone doesn't prove the model's gotten any less accurate. It's a prompt to look closer, not a verdict.*

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5 · Invented counts

## 20 / 55 Concept drift: a different relationship

Block 1 / Monitoring & degradation • Simon




**CUES**

- Presenter: **Simon**.
- Animated build: click 1 time to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

**TO SAY**

- The same inputs can have different outcomes while the deployed model keeps its earlier boundary.
- **What changes**: churn risk given the same valid input information.
- **Bank example**: new fees make active customers leave more often.
- **Evidence**: link predictions to mature, reliable churn outcomes.
- **Model status**: M1 stays fixed until a separately evaluated release changes it.
- *At our bank,* the two panels use the same 15 fictional feature positions. After the hypothetical change, some previously staying customers churn.
- *A missing field is not concept drift; a statistical alarm is not a retraining approval.*

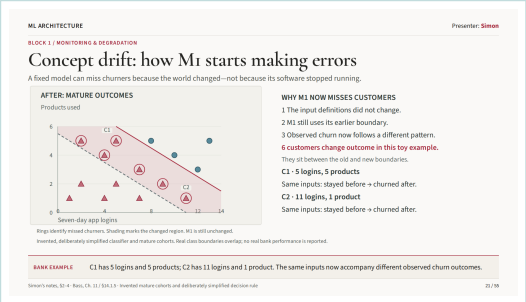
**SPOKEN EXAMPLE — practice this out loud**

*Concept drift is a different animal: the inputs look exactly the same, but the relationship between those inputs and the outcome has shifted. Say the bank introduces a new fee, and now active customers start leaving at a higher rate than before — same engagement pattern, different result. We can only really see this once we have mature, trustworthy churn outcomes to compare against — not right away. And through all of this, M1 itself hasn't changed; it keeps using its old decision boundary until a properly evaluated new release replaces it. One thing concept drift is not: a single missing field, and it's definitely not an automatic green light to retrain.*

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5 · Schematic relationships

## 21 / 55 Concept drift: how M1 starts making errors

Block 1 / Monitoring & degradation • Simon




### CUES

- Presenter: **Simon**.
- Animated build: click 1 time to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.
- Adaptation note on file for this slide; do not read aloud unless asked.

### TO SAY

- A fixed model can miss churners because the world changed—not because its software stopped running.
- **Inputs stay fixed**: the two cohorts use identical feature positions.
- **M1 stays fixed**: its earlier decision boundary is unchanged.
- **Outcomes change**: a different relationship creates missed churners.
- **Evidence**: the example uses mature labels, not outcomes already known on scoring day.
- *At our bank,* c1 has 5 logins and 5 products; C2 has 11 logins and 1 product. The same inputs now accompany different observed churn outcomes.
- *Changed labels reveal the mismatch. Investigate causes, evaluate candidates, and use controlled release rather than automatic promotion.*

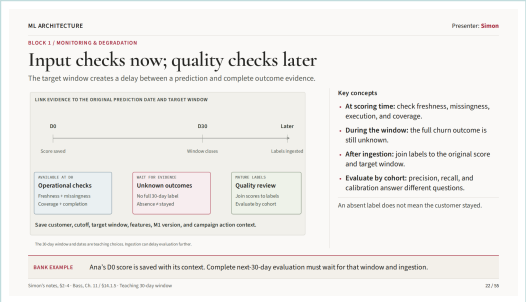
### SPOKEN EXAMPLE — practice this out loud

*Let's make concept drift concrete. Picture two customers with exactly the same inputs as before — say, five logins and five products used — but now, where they used to stay, they churn. The inputs haven't moved. M1 hasn't moved either; it's still using the boundary it learned originally. What's moved is the real-world relationship between those inputs and what customers actually do. That mismatch is exactly where a fixed model starts quietly making mistakes — not because anything crashed, but because the world underneath it kept moving. Once we see this kind of evidence in mature labels, the next step is to investigate the cause, properly evaluate a candidate fix, and release it through a controlled process — never promote something automatically.*

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5 · Invented mature cohorts and deliberately simplified decision rule

## 22 / 55 Input checks now; quality checks later

Block 1 / Monitoring & degradation • Simon




### CUES

- Presenter: **Simon**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- The target window creates a delay between a prediction and complete outcome evidence.
- **At scoring time**: check freshness, missingness, execution, and coverage.
- **During the window**: the full churn outcome is still unknown.
- **After ingestion**: join labels to the original score and target window.
- **Evaluate by cohort**: precision, recall, and calibration answer different questions.
- Ana's D0 score is saved with its context. Complete next-30-day evaluation must wait for that window and ingestion.
- *An absent label does not mean the customer stayed.*

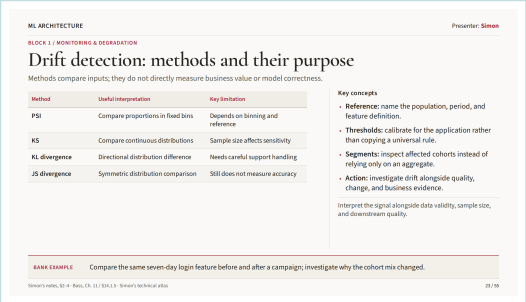
### SPOKEN EXAMPLE — practice this out loud

*There's an unavoidable delay baked into how we evaluate churn predictions: we define a target window, so by definition the full outcome isn't known yet when we score someone. At scoring time, all we can check is whether the inputs were fresh, complete, and the pipeline ran correctly. Only once that window closes and the real labels arrive can we join them back to the original score and properly evaluate precision, recall, and calibration by cohort. Think of Ana's score on day zero: it's saved with its full context, but we genuinely cannot evaluate it properly until 30 days later. And a missing label at that point does not mean she stayed — it might just mean the data hasn't arrived yet.*

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5 · Teaching 30-day window

## 23 / 55 Drift detection: methods and their purpose

Block 1 / Monitoring & degradation • Simon




### CUES

- Presenter: **Simon**.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Methods compare inputs; they do not directly measure business value or model correctness.
- **Reference**: name the population, period, and feature definition.
- **Thresholds**: calibrate for the application rather than copying a universal rule.
- **Segments**: inspect affected cohorts instead of relying only on an aggregate.
- **Action**: investigate drift alongside quality, change, and business evidence.
- *At our bank,* compare the same seven-day login feature before and after a campaign; investigate why the cohort mix changed.
- *Interpret the signal alongside data validity, sample size, and downstream quality.*

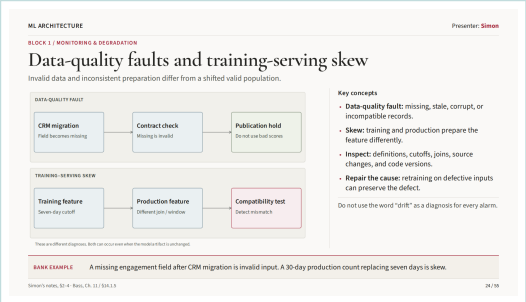
### SPOKEN EXAMPLE — practice this out loud

*Drift-detection methods are good at one narrow thing: comparing input distributions. They don’t, on their own, tell you anything about business value or whether the model itself is still correct. To use them well, you need to name your reference population and time period clearly, calibrate your thresholds for your own situation rather than copying someone else’s rule of thumb, and look at specific affected segments rather than trusting a single aggregate number. If we compare the same seven-day login feature before and after a marketing campaign and see a shift, the right next move is to go find out why the customer mix actually changed — the statistic alone won’t tell you.*

Reading anchor: Simon’s notes, §2–4 · Bass, Ch. 11 / §14.1.5 · Simon’s technical atlas

## 24 / 55 Data-quality faults and training-serving skew

Block 1 / Monitoring & degradation • Simon




### CUES

- Presenter: **Simon**.
- Animated build: click 3 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Invalid data and inconsistent preparation differ from a shifted valid population.
- **Data-quality fault**: missing, stale, corrupt, or incompatible records.
- **Skew**: training and production prepare the feature differently.
- **Inspect**: definitions, cutoffs, joins, source changes, and code versions.
- **Repair the cause**: retraining on defective inputs can preserve the defect.
- *At our bank,* a missing engagement field after CRM migration is invalid input. A 30-day production count replacing seven days is skew.
- *Do not use the word “drift” as a diagnosis for every alarm.*

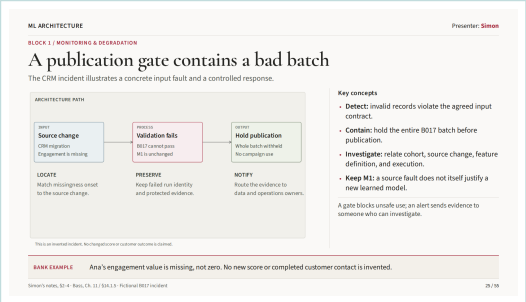
### SPOKEN EXAMPLE — practice this out loud

*It’s worth drawing a sharp line between two things that often get lumped together. A data-quality fault is something simply broken — missing, stale, or corrupted records. Training-serving skew is subtler: training and production are preparing the exact same feature in two different ways. A missing engagement value after a CRM migration is a quality fault, plain and simple. Using a 30-day activity count in production when training used seven days — that’s skew. In both cases, the fix is to repair the actual cause. If you retrain on data that’s still broken, you just teach the model to be wrong in a new, more confident way. Don’t let ‘drift’ become the catch-all word you reach for every time an alarm fires.*

Reading anchor: Simon’s notes, §2–4 · Bass, Ch. 11 / §14.1.5

## 25 / 55 A publication gate contains a bad batch

Block 1 / Monitoring & degradation • Simon




### CUES

- Presenter: **Simon**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- The CRM incident illustrates a concrete input fault and a controlled response.
- **Detect**: invalid records violate the agreed input contract.
- **Contain**: hold the entire B017 batch before publication.
- **Investigate**: relate cohort, source change, feature definition, and execution.
- **Keep M1**: a source fault does not itself justify a new learned model.
- Ana's engagement value is missing, not zero. No new score or completed customer contact is invented.
- *A gate blocks unsafe use; an alert sends evidence to someone who can investigate.*

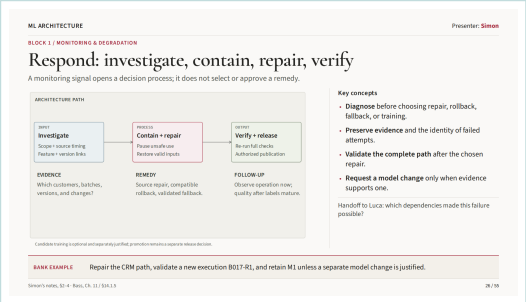
### SPOKEN EXAMPLE — practice this out loud

*Let's walk through a real incident, even if the details are invented for teaching. After the CRM migration, some records come in with engagement missing — not zero, genuinely missing — and that violates our agreed input contract. The detection is automatic. What happens next is containment: we hold the entire batch, which we'll call B017, before it ever reaches the campaign. Then we investigate — how does this cohort, this source change, and this feature definition actually connect? And importantly, we keep M1 exactly as it is. A broken input source, on its own, is never a reason to reach for a new model. The gate's job is to block unsafe use; the alert's job is to get the evidence to someone who can actually dig in.*

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5 · Fictional B017 incident

## 26 / 55 Respond: investigate, contain, repair, verify

Block 1 / Monitoring & degradation • Simon




### CUES

- Presenter: **Simon**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- A monitoring signal opens a decision process; it does not select or approve a remedy.
- **Diagnose** before choosing repair, rollback, fallback, or training.
- **Preserve evidence** and the identity of failed attempts.
- **Validate the complete path** after the chosen repair.
- **Request a model change** only when evidence supports one.
- *At our bank,* repair the CRM path, validate a new execution B017-R1, and retain M1 unless a separate model change is justified.
- *Handoff to Luca: which dependencies made this failure possible?*

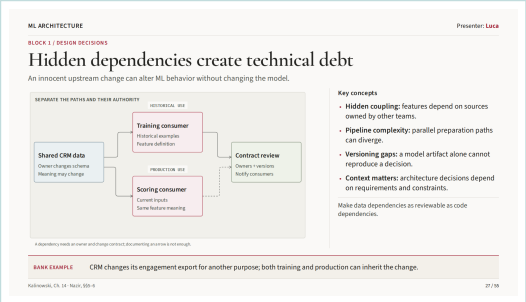
### SPOKEN EXAMPLE — practice this out loud

*A monitoring signal is the start of a decision process — it never makes the decision for you. The first step is always to diagnose before reaching for a fix, whether that's a repair, a rollback, a fallback, or retraining. Along the way, we preserve the evidence of exactly what failed and how. Once we've chosen a repair, we validate the whole path end to end, not just the piece we touched. And a model change only gets requested when the evidence actually supports one. In our incident, that means: repair the CRM path, validate a brand-new execution we'll call B017-R1, and keep M1 unless there's a separate, justified reason to change it. From here, Luca's going to pick up the question of what let this failure happen in the first place.*

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5

## 27 / 55 Hidden dependencies create technical debt

Block 1 / Design decisions • Luca




**CUES**

- New block: **Block 1 / Design decisions**.
- Presenter: **Luca**.
- Animated build: click 3 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

**TO SAY**

- An innocent upstream change can alter ML behavior without changing the model.
- **Hidden coupling**: features depend on sources owned by other teams.
- **Pipeline complexity**: parallel preparation paths can diverge.
- **Versioning gaps**: a model artifact alone cannot reproduce a decision.
- **Context matters**: architecture decisions depend on requirements and constraints.
- *At our bank,* cRM changes its engagement export for another purpose; both training and production can inherit the change.
- *Make data dependencies as reviewable as code dependencies.*

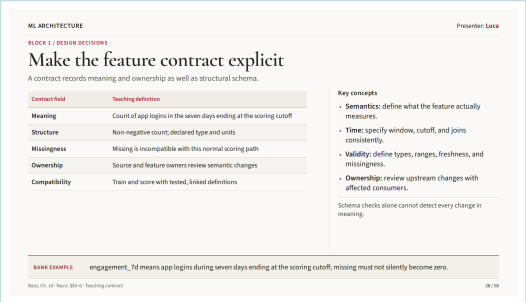
**SPOKEN EXAMPLE — practice this out loud**

*Here's an uncomfortable truth: a completely innocent change upstream can quietly change how your model behaves, without anyone touching the model at all. Features often depend on data sources owned by other teams entirely. Parallel preparation paths — one for training, one for production — can drift apart from each other over time. And a model artifact on its own can't reproduce a past decision if you don't also know exactly which version of everything else was involved. In our story, the CRM team changes its engagement export for reasons that have nothing to do with the churn model — and both training and production inherit that change anyway. The lesson is simple: treat your data dependencies with the same seriousness you'd treat a code dependency.*

Reading anchor: Kalinowski, Ch. 14 · Nazir, §§5–6

## 28 / 55 Make the feature contract explicit

Block 1 / Design decisions • Luca




### CUES

- Presenter: **Luca**.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- A contract records meaning and ownership as well as structural schema.
- **Semantics**: define what the feature actually measures.
- **Time**: specify window, cutoff, and joins consistently.
- **Validity**: define types, ranges, freshness, and missingness.
- **Ownership**: review upstream changes with affected consumers.
- *At our bank,* engagement_7d means app logins during seven days ending at the scoring cutoff; missing must not silently become zero.
- *Schema checks alone cannot detect every change in meaning.*

### SPOKEN EXAMPLE — practice this out loud

*A good feature contract records more than just a data type — it records meaning. What does this feature actually measure? Over what time window, with what cutoff? What counts as valid, and what should happen when a value is missing? And who do you talk to before that upstream source changes? Take our feature, engagement_7d: it means app logins over the seven days ending at the scoring cutoff — and a missing value must never silently become a zero, because that's a completely different claim about the customer. A schema check alone would never have caught a change in what the feature actually means.*

Reading anchor: Bass, Ch. 14 · Nazir, §§5–6 · Teaching contract

## 29 / 55 Separate responsibilities where change happens

Block 1 / Design decisions • Luca




### CUES

- Presenter: **Luca**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Clear interfaces let different concerns evolve without unnecessary relearning.
- **Single responsibility**: identify each module's reason to change.
- **Interfaces**: specify data and behavior at each boundary.
- **Dependency injection**: provide implementations through explicit contracts.
- **Deployment**: logical modules do not automatically require separate services.
- *At our bank,* a connector fix changes preparation; a contact-frequency rule changes campaign policy; a new M2 changes inference.
- *Separate concerns first; choose deployment units from the actual trade-offs.*

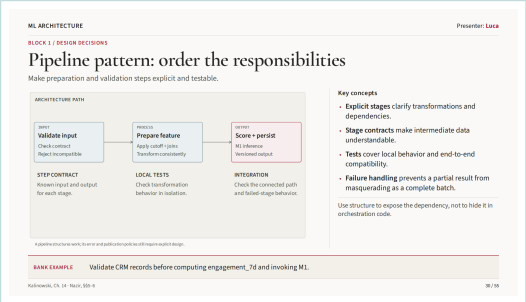
### SPOKEN EXAMPLE — practice this out loud

*Clean interfaces let different parts of a system evolve on their own schedules, without forcing everything else to change along with them. Each module should have one clear reason to change, and the data and behavior crossing each boundary should be explicitly specified. Think about three very different kinds of change at our bank: fixing a connector changes how data gets prepared; changing a contact-frequency rule changes campaign policy; releasing a new M2 changes inference. None of those three should force the other two to change. And just because you've separated these concerns logically doesn't mean you're obligated to deploy them as three separate services — that's a different decision, made on its own merits.*

Reading anchor: Kalinowski, Ch. 14 · Nazir, §§5–6

## 30 / 55 Pipeline pattern: order the responsibilities

Block 1 / Design decisions • Luca




**CUES**

- Presenter: **Luca**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

**TO SAY**

- Make preparation and validation steps explicit and testable.
- **Explicit stages** clarify transformations and dependencies.
- **Stage contracts** make intermediate data understandable.
- **Tests** cover local behavior and end-to-end compatibility.
- **Failure handling** prevents a partial result from masquerading as a complete batch.
- *At our bank,* validate CRM records before computing engagement_7d and invoking M1.
- *Use structure to expose the dependency, not to hide it in orchestration code.*

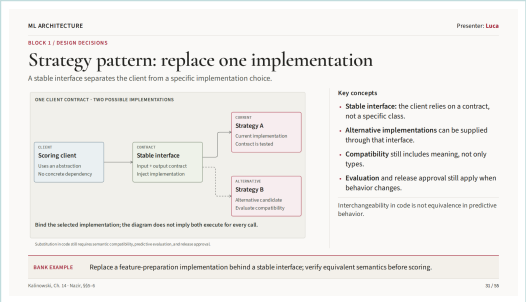
**SPOKEN EXAMPLE — practice this out loud**

*Structuring your pipeline as explicit stages makes the transformations and their dependencies visible, instead of hidden inside one tangled script. Each stage should have a contract so the data passing between them stays understandable, and you want tests both at the stage level and across the whole pipeline. Failure handling matters too — you never want a partial result quietly passing itself off as a complete batch. Concretely: CRM records get validated before we ever compute engagement_7d or hand anything to M1. The goal of this structure isn't to hide the dependency on upstream data — it's to make it visible enough that you can actually reason about it.*

Reading anchor: Kalinowski, Ch. 14 · Nazir, §§5–6

## 31 / 55 Strategy pattern: replace one implementation

Block 1 / Design decisions • Luca




### CUES

- Presenter: **Luca**.
- Animated build: click 3 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- A stable interface separates the client from a specific implementation choice.
- **Stable interface**: the client relies on a contract, not a specific class.
- **Alternative implementations** can be supplied through that interface.
- **Compatibility** still includes meaning, not only types.
- **Evaluation** and release approval still apply when behavior changes.
- *At our bank,* replace a feature-preparation implementation behind a stable interface; verify equivalent semantics before scoring.
- *Interchangeability in code is not equivalence in predictive behavior.*

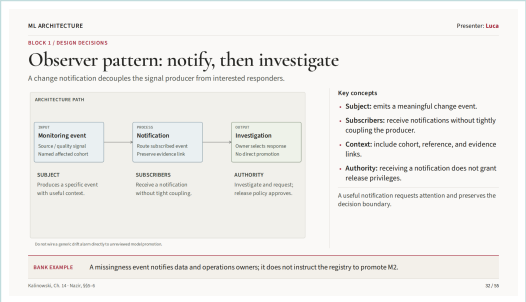
### SPOKEN EXAMPLE — practice this out loud

*The Strategy pattern lets a client code against a stable interface, while the actual implementation behind that interface can be swapped out. So you might replace how a feature gets prepared, behind an interface the rest of the system never has to know changed. But here's the catch for ML specifically: compatibility has to include meaning, not just matching types. Two implementations can both compile and both return a float, and still mean something subtly different. So even when you swap an implementation this way, evaluation and release approval still apply — being interchangeable in code is not the same as being equivalent in predictive behavior.*

Reading anchor: Kalinowski, Ch. 14 · Nazir, §§5–6

## 32 / 55 Observer pattern: notify, then investigate

Block 1 / Design decisions • Luca




### CUES

- Presenter: **Luca**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- A change notification decouples the signal producer from interested responders.
- **Subject**: emits a meaningful change event.
- **Subscribers**: receive notifications without tightly coupling the producer.
- **Context**: include cohort, reference, and evidence links.
- **Authority**: receiving a notification does not grant release privileges.
- *At our bank,* a missingness event notifies data and operations owners; it does not instruct the registry to promote M2.
- *A useful notification requests attention and preserves the decision boundary.*

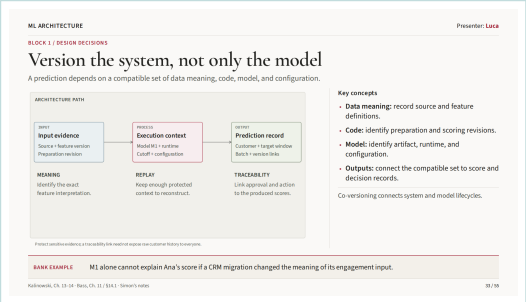
### SPOKEN EXAMPLE — practice this out loud

*The Observer pattern decouples whoever notices a problem from whoever needs to act on it. A subject emits a meaningful event when something changes; subscribers receive that notification without being tightly wired to whoever produced it. The notification should carry real context — which cohort, which reference, what evidence backs it up. And here's the important boundary: receiving a notification is not the same as having authority to act on it. A missingness event tells the data and operations owners something's wrong; it does not, by itself, instruct anything to promote M2 into production. A good notification asks for attention — it doesn't quietly grant permission.*

Reading anchor: Kalinowski, Ch. 14 · Nazir, §§5–6

## 33 / 55 Version the system, not only the model

Block 1 / Design decisions • Luca




**CUES**

- Presenter: **Luca**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

**TO SAY**

- A prediction depends on a compatible set of data meaning, code, model, and configuration.
- **Data meaning**: record source and feature definitions.
- **Code**: identify preparation and scoring revisions.
- **Model**: identify artifact, runtime, and configuration.
- **Outputs**: connect the compatible set to score and decision records.
- *At our bank,* m1 alone cannot explain Ana's score if a CRM migration changed the meaning of its engagement input.
- *Co-versioning connects system and model lifecycles.*

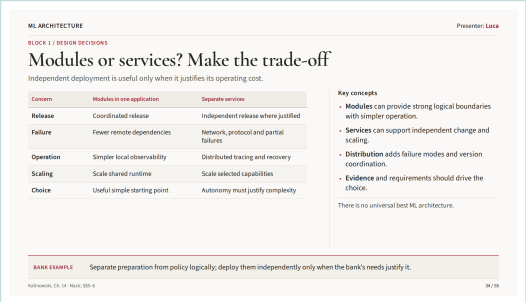
**SPOKEN EXAMPLE — practice this out loud**

*A single prediction actually depends on a whole compatible set of things: what the data meant at the time, which code prepared it, which model scored it, and what configuration was active. Versioning the model alone misses most of that picture. If a CRM migration quietly changed what 'engagement' means, then knowing it was M1 that produced Ana's score tells you almost nothing useful on its own — you need the data definition that went with it too. Co-versioning is what actually connects your system's lifecycle to your model's lifecycle, instead of treating them as two separate stories.*

Reading anchor: Kalinowski, Ch. 13–14 · Bass, Ch. 11 / §14.1 · Simon's notes

## 34 / 55 Modules or services? Make the trade-off

Block 1 / Design decisions • Luca




### CUES

- Presenter: **Luca**.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Independent deployment is useful only when it justifies its operating cost.
- **Modules** can provide strong logical boundaries with simpler operation.
- **Services** can support independent change and scaling.
- **Distribution** adds failure modes and version coordination.
- **Evidence** and requirements should drive the choice.
- *At our bank,* separate preparation from policy logically; deploy them independently only when the bank's needs justify it.
- *There is no universal best ML architecture.*

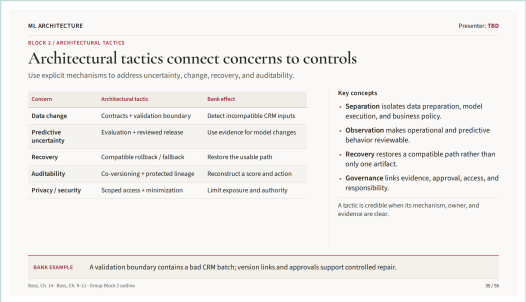
### SPOKEN EXAMPLE — practice this out loud

*Splitting something into an independently deployable service is only worth it when that independence actually pays for its own operational cost. Modules give you strong logical boundaries with much simpler operations. Services let you scale and change pieces independently — but distribution brings its own failure modes and the headache of keeping versions coordinated across the boundary. At the bank, we'd separate feature preparation from policy logic cleanly as modules first, and only turn either into a separate service once there's real evidence it's worth the operational cost. There's no single best architecture for every ML system — the right answer depends on your actual constraints.*

Reading anchor: Kalinowski, Ch. 14 · Nazir, §§5–6

## 35 / 55 Architectural tactics connect concerns to controls

Block 2 / Architectural tactics • TBD




### CUES

- New block: **Block 2 / Architectural tactics**.
- Presenter: **TBD**.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Use explicit mechanisms to address uncertainty, change, recovery, and auditability.
- **Separation** isolates data preparation, model execution, and business policy.
- **Observation** makes operational and predictive behavior reviewable.
- **Recovery** restores a compatible path rather than only one artifact.
- **Governance** links evidence, approval, access, and responsibility.
- *At our bank,* a validation boundary contains a bad CRM batch; version links and approvals support controlled repair.
- *A tactic is credible when its mechanism, owner, and evidence are clear.*

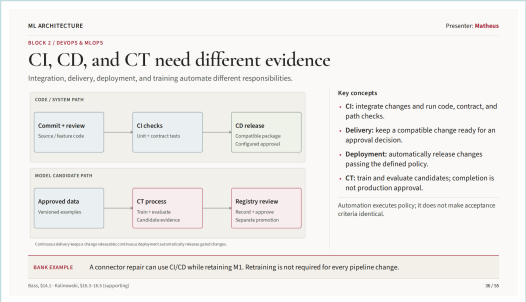
### SPOKEN EXAMPLE — practice this out loud

*Let’s name the tactics we’ve actually been using throughout, now that Block 2 starts. Separation keeps data preparation, model execution, and business policy from tangling together. Observation is what makes both operational and predictive behavior something you can actually review. Recovery means restoring a whole compatible path, not just swapping back one file. And governance ties evidence, approval, access, and responsibility together into something accountable. In our CRM incident, a validation boundary is what contained the bad batch, and version links plus approvals are what made the repair trustworthy. A tactic earns its keep when you can point to its mechanism, its owner, and its evidence — not just gesture at a diagram.*

Reading anchor: Bass, Ch. 14 · Bass, Ch. 9–11 · Group Block 2 outline

## 36 / 55 CI, CD, and CT need different evidence

Block 2 / DevOps & MLOps • Matheus




### CUES

- New block: **Block 2 / DevOps & MLOps**.
- Presenter: **Matheus**.
- Animated build: click 3 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Integration, delivery, deployment, and training automate different responsibilities.
- **CI**: integrate changes and run code, contract, and path checks.
- **Delivery**: keep a compatible change ready for an approval decision.
- **Deployment**: automatically release changes passing the defined policy.
- **CT**: train and evaluate candidates; completion is not production approval.
- *At our bank,* a connector repair can use CI/CD while retaining M1. Retraining is not required for every pipeline change.
- *Automation executes policy; it does not make acceptance criteria identical.*

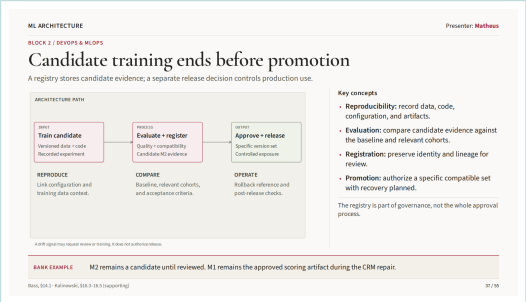
### SPOKEN EXAMPLE — practice this out loud

*CI, CD, and CT sound like one pipeline, but they automate genuinely different responsibilities. Continuous integration runs your code, contract, and path checks whenever something changes. Continuous delivery keeps a compatible change ready, waiting on an actual approval decision. Continuous deployment automatically releases anything that passes a defined policy. And continuous training trains and evaluates candidate models — but finishing that training run is not the same thing as being approved for production. A connector repair, for instance, can go through CI/CD perfectly well while M1 stays exactly as it is — fixing the pipeline doesn't require retraining the model.*

Reading anchor: Bass, §14.1 · Kalinowski, §16.3–16.5 (supporting)

## 37 / 55 Candidate training ends before promotion

Block 2 / DevOps & MLOps • Matheus




### CUES

- Presenter: **Matheus**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- A registry stores candidate evidence; a separate release decision controls production use.
- **Reproducibility**: record data, code, configuration, and artifacts.
- **Evaluation**: compare candidate evidence against the baseline and relevant cohorts.
- **Registration**: preserve identity and lineage for review.
- **Promotion**: authorize a specific compatible set with recovery planned.
- *At our bank,* m2 remains a candidate until reviewed. M1 remains the approved scoring artifact during the CRM repair.
- *The registry is part of governance, not the whole approval process.*

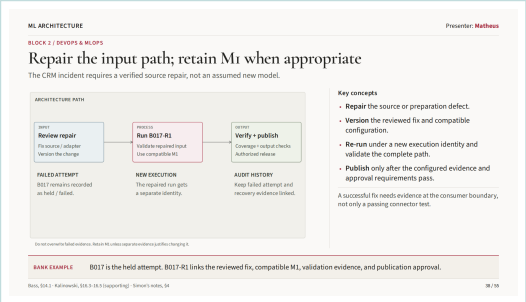
### SPOKEN EXAMPLE — practice this out loud

*A model registry's job is to preserve candidate evidence — it is not, by itself, the approval. You need reproducibility: the exact data, code, configuration, and artifacts that produced a candidate. You evaluate it against both the current baseline and the cohorts that actually matter. Registration preserves identity and lineage so it can be properly reviewed. And promotion is a distinct act — authorizing a specific, compatible set of things for production, with a recovery plan already in place. Concretely: M2 sits in the registry as a candidate until it's actually reviewed, while M1 stays the one approved artifact scoring customers throughout the CRM repair.*

Reading anchor: Bass, §14.1 · Kalinowski, §16.3–16.5 (supporting)

## 38 / 55 Repair the input path; retain M1 when appropriate

Block 2 / DevOps & MLOps • Matheus




### CUES

- Presenter: **Matheus**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- The CRM incident requires a verified source repair, not an assumed new model.
- **Repair** the source or preparation defect.
- **Version** the reviewed fix and compatible configuration.
- **Re-run** under a new execution identity and validate the complete path.
- **Publish** only after the configured evidence and approval requirements pass.
- *At our bank,* b017 is the held attempt. B017-R1 links the reviewed fix, compatible M1, validation evidence, and publication approval.
- *A successful fix needs evidence at the consumer boundary, not only a passing connector test.*

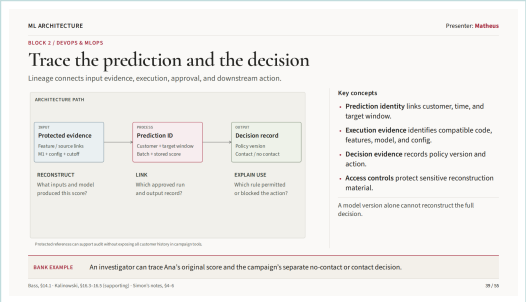
### SPOKEN EXAMPLE — practice this out loud

*When the CRM incident needs fixing, what's actually required is a verified repair to the input path — not a reflexive jump to training a new model. First you repair the actual source or preparation defect. Then you version the reviewed fix together with its compatible configuration. You re-run it under a brand-new execution identity, and you validate the complete path end to end. Only then do you publish, and only once the evidence and approval requirements you've already defined are satisfied. B017 is the batch we held back; B017-R1 is the new execution that links the reviewed fix, the still-compatible M1, real validation evidence, and an actual publication approval. A passing connector test on its own was never going to be enough evidence.*

Reading anchor: Bass, §14.1 · Kalinowski, §16.3–16.5 (supporting) · Simon's notes, §4

## 39 / 55 Trace the prediction and the decision

Block 2 / DevOps & MLOps • Matheus




**CUES**

- Presenter: **Matheus**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

**TO SAY**

- Lineage connects input evidence, execution, approval, and downstream action.
- **Prediction identity** links customer, time, and target window.
- **Execution evidence** identifies compatible code, features, model, and config.
- **Decision evidence** records policy version and action.
- **Access controls** protect sensitive reconstruction material.
- *At our bank,* an investigator can trace Ana's original score and the campaign's separate no-contact or contact decision.
- *A model version alone cannot reconstruct the full decision.*

**SPOKEN EXAMPLE — practice this out loud**

*Lineage is what lets you connect input evidence, execution, approval, and whatever action followed — all the way through. A prediction's identity links it to a specific customer, a specific time, and a specific target window. Execution evidence tells you exactly which code, features, model, and configuration were compatible and in play. Decision evidence records which policy version applied and what action resulted. And because this evidence is sensitive, access to it needs its own controls. With this in place, an investigator really can trace back Ana's original score and, separately, the campaign's decision to contact her or not. A model version number alone was never going to reconstruct that whole story.*

Reading anchor: Bass, §14.1 · Kalinowski, §16.3–16.5 (supporting) · Simon's notes, §4–6

## 40 / 55 Recovery restores a compatible path

Block 2 / DevOps & MLOps • Matheus




### CUES

- Presenter: **Matheus**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Rehearse restoration before release, and verify it at the consumer boundary.
- **Know the target**: identify the last compatible release or validated fallback.
- **Rehearse**: test the recovery procedure before relying on it.
- **Restore the set**: include data meaning, runtime, and configuration.
- **Verify**: inspect stored scores and actual consumer behavior.
- *At our bank,* rolling M1 back cannot repair a CRM field that is still missing; the input path must be restored.
- *Handoff to security: who may read, change, approve, or operate these paths?*

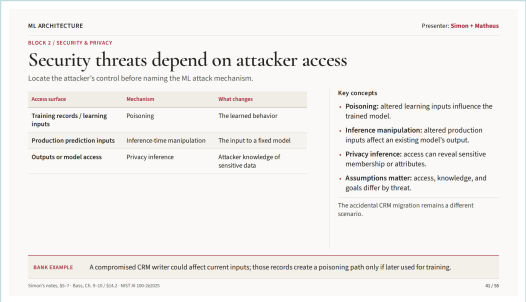
### SPOKEN EXAMPLE — practice this out loud

*Recovery means restoring a compatible path — and you should never be testing that path for the first time during an actual incident. First, know your target: what's the last compatible release, or a validated fallback you can trust? Rehearse the actual recovery procedure before you ever need it for real. When you restore, restore the whole set — data meaning, runtime, and configuration together, not just one piece. And afterward, verify by checking real stored scores and real consumer behavior, not just that a command ran successfully. Here's the trap in our incident: rolling M1 back does absolutely nothing to fix a CRM field that's still missing. The input path itself is what needs restoring. From here, we hand off to security — who's actually allowed to read, change, approve, or operate any of these paths?*

Reading anchor: Breck, infrastructure tests 6–7 · Simon's notes, §4

## 41 / 55 Security threats depend on attacker access

Block 2 / Security & privacy • Simon + Matheus




**CUES**

- New block: **Block 2 / Security & privacy**.
- Presenter: **Simon + Matheus**.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

**TO SAY**

- Locate the attacker's control before naming the ML attack mechanism.
- **Poisoning**: altered learning inputs influence the trained model.
- **Inference manipulation**: altered production inputs affect an existing model's output.
- **Privacy inference**: access can reveal sensitive membership or attributes.
- **Assumptions matter**: access, knowledge, and goals differ by threat.
- *At our bank,* a compromised CRM writer could affect current inputs; those records create a poisoning path only if later used for training.
- *The accidental CRM migration remains a different scenario.*

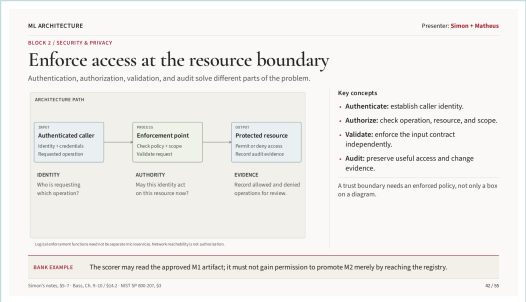
**SPOKEN EXAMPLE — practice this out loud**

*Before naming an attack by its textbook mechanism, locate exactly what the attacker can actually touch. Poisoning means altered inputs make it into training and corrupt the model that gets learned. Inference manipulation means altered inputs affect an already-trained model's output, at prediction time. Privacy inference means access to outputs or behavior reveals something sensitive about who's in the data. Each of these rests on different assumptions about access, knowledge, and goals. Picture a compromised CRM writer: it could affect current production inputs right away — but it only becomes a poisoning attack if those same altered records later get used to train a future model. And to be clear, our earlier CRM migration incident was accidental, not an attack at all — a genuinely different scenario.*

Reading anchor: Simon's notes, §5–7 · Bass, Ch. 9–10 / §14.2 · NIST AI 100-2e2025

## 42 / 55 Enforce access at the resource boundary

Block 2 / Security & privacy • Simon + Matheus




### CUES

- Presenter: **Simon + Matheus**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Authentication, authorization, validation, and audit solve different parts of the problem.
- **Authenticate**: establish caller identity.
- **Authorize**: check operation, resource, and scope.
- **Validate**: enforce the input contract independently.
- **Audit**: preserve useful access and change evidence.
- *At our bank,* the scorer may read the approved M1 artifact; it must not gain permission to promote M2 merely by reaching the registry.
- *A trust boundary needs an enforced policy, not only a box on a diagram.*

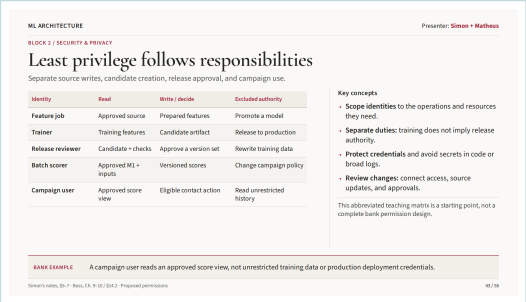
### SPOKEN EXAMPLE — practice this out loud

*A trust boundary only matters if it's actually enforced — not just drawn as a box on a diagram. Authentication establishes who's calling. Authorization checks whether that caller can do this specific operation on this specific resource. Validation enforces the input contract completely independently of who's asking. And audit preserves a real trail of access and change. Concretely: the scoring service should be allowed to read the approved M1 artifact — but reaching the model registry should never, by itself, grant it permission to promote M2 into production. Those are different operations and need different permissions.*

Reading anchor: Simon's notes, §5–7 · Bass, Ch. 9–10 / §14.2 · NIST SP 800-207, §3

## 43 / 55 Least privilege follows responsibilities

Block 2 / Security & privacy • Simon + Matheus




### CUES

- Presenter: **Simon + Matheus**.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Separate source writes, candidate creation, release approval, and campaign use.
- **Scope identities** to the operations and resources they need.
- **Separate duties**: training does not imply release authority.
- **Protect credentials** and avoid secrets in code or broad logs.
- **Review changes**: connect access, source updates, and approvals.
- *At our bank,* a campaign user reads an approved score view, not unrestricted training data or production deployment credentials.
- *This abbreviated teaching matrix is a starting point, not a complete bank permission design.*

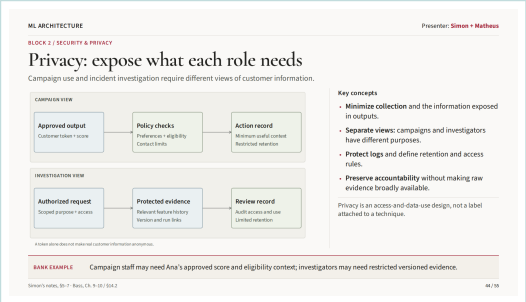
### SPOKEN EXAMPLE — practice this out loud

*Least privilege means scoping every identity to exactly the operations and resources it actually needs — nothing more. Separate the duties clearly: training a candidate model should never, on its own, carry the authority to release it. Protect credentials properly, and keep secrets out of code and out of broad-access logs. And connect access changes, source updates, and approvals so they're reviewable together. A campaign user, for example, should only be able to read an approved score view — not raw training data, and definitely not production deployment credentials. This simplified matrix is meant as a starting point for thinking it through, not a complete real-world permission design for an actual bank.*

Reading anchor: Simon's notes, §5–7 · Bass, Ch. 9–10 / §14.2 · Proposed permissions

## 44 / 55 Privacy: expose what each role needs

Block 2 / Security & privacy • Simon + Matheus




### CUES

- Presenter: **Simon + Matheus**.
- Animated build: click 3 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Campaign use and incident investigation require different views of customer information.
- **Minimize collection** and the information exposed in outputs.
- **Separate views**: campaigns and investigators have different purposes.
- **Protect logs** and define retention and access rules.
- **Preserve accountability** without making raw evidence broadly available.
- *At our bank,* campaign staff may need Ana's approved score and eligibility context; investigators may need restricted versioned evidence.
- *Privacy is an access-and-data-use design, not a label attached to a technique.*

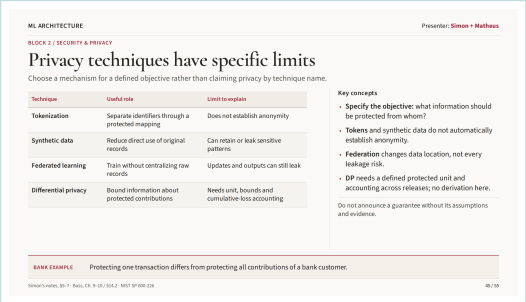
### SPOKEN EXAMPLE — practice this out loud

*Campaign work and incident investigation genuinely need different views into the same customer data. Minimize what you collect and what ends up exposed in any output. Keep those views separate by purpose — what a campaign team needs is not what an investigator needs. Protect logs with clear retention and access rules. And preserve accountability without making raw evidence broadly available to everyone. Campaign staff might reasonably need Ana's approved score and her eligibility status; an investigator looking into an incident might need much more restricted, fully versioned evidence. Privacy, in this sense, is a design decision about access and data use — not a label you attach after the fact because you used some particular technique.*

Reading anchor: Simon's notes, §5–7 · Bass, Ch. 9–10 / §14.2

## 45 / 55 Privacy techniques have specific limits

Block 2 / Security & privacy • Simon + Matheus




### CUES

- Presenter: **Simon + Matheus**.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Choose a mechanism for a defined objective rather than claiming privacy by technique name.
- **Specify the objective**: what information should be protected from whom?
- **Tokens** and synthetic data do not automatically establish anonymity.
- **Federation** changes data location, not every leakage risk.
- **DP** needs a defined protected unit and accounting across releases; no derivation here.
- *At our bank,* protecting one transaction differs from protecting all contributions of a bank customer.
- *Do not announce a guarantee without its assumptions and evidence.*

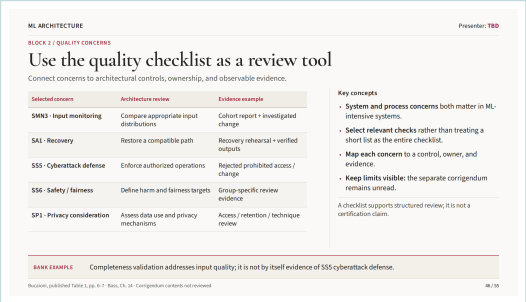
### SPOKEN EXAMPLE — practice this out loud

*Before reaching for a specific privacy technique, name exactly what you're trying to protect, and from whom. Tokenizing an identifier or generating synthetic data doesn't automatically mean you've achieved anonymity — those are separate claims that need their own evidence. Federation changes where data physically lives; it doesn't eliminate every way information could leak. And differential privacy needs a clearly defined protected unit and careful accounting across every release — we won't derive the mathematics here, but the idea matters: protecting one single transaction is a very different problem from protecting everything one customer ever contributed. Never announce a privacy guarantee without naming its actual assumptions and the evidence behind it.*

Reading anchor: Simon's notes, §5–7 · Bass, Ch. 9–10 / §14.2 · NIST SP 800-226

## 46 / 55 Use the quality checklist as a review tool

Block 2 / Quality concerns • TBD




**CUES**

- New block: **Block 2 / Quality concerns**.
- Presenter: **TBD**.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

**TO SAY**

- Connect concerns to architectural controls, ownership, and observable evidence.
- **System and process concerns** both matter in ML-intensive systems.
- **Select relevant checks** rather than treating a short list as the entire checklist.
- **Map each concern** to a control, owner, and evidence.
- **Keep limits visible**: the separate corrigendum remains unread.
- *At our bank,* completeness validation addresses input quality; it is not by itself evidence of SS5 cyberattack defense.
- *A checklist supports structured review; it is not a certification claim.*

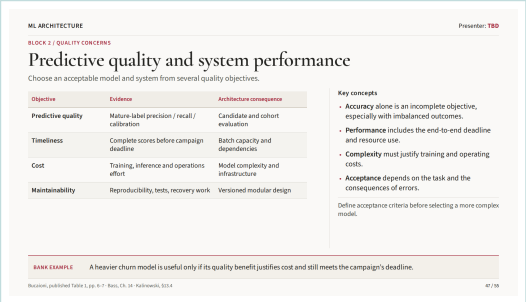
**SPOKEN EXAMPLE — practice this out loud**

*Think of a quality checklist as a tool for structured review, not a certificate you can wave around. Both system concerns and process concerns matter for ML-heavy systems. Pick the checks that are actually relevant to you, rather than treating a short published list as the entire checklist. And for each concern you do pick, map it to a real control, a real owner, and real evidence. As one example: completeness validation is genuinely good evidence about input quality — but it says nothing at all, by itself, about defense against a sophisticated cyberattack. Be honest about what a checklist item actually proves, and keep its limits visible.*

Reading anchor: Bucaioni, published Table 1, pp. 6–7 · Bass, Ch. 14 · Corrigendum contents not reviewed

## 47 / 55 Predictive quality and system performance

Block 2 / Quality concerns • TBD




**CUES**

- Presenter: **TBD**.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

**TO SAY**

- Choose an acceptable model and system from several quality objectives.
- **Accuracy** alone is an incomplete objective, especially with imbalanced outcomes.
- **Performance** includes the end-to-end deadline and resource use.
- **Complexity** must justify training and operating costs.
- **Acceptance** depends on the task and the consequences of errors.
- *At our bank,* a heavier churn model is useful only if its quality benefit justifies cost and still meets the campaign's deadline.
- *Define acceptance criteria before selecting a more complex model.*

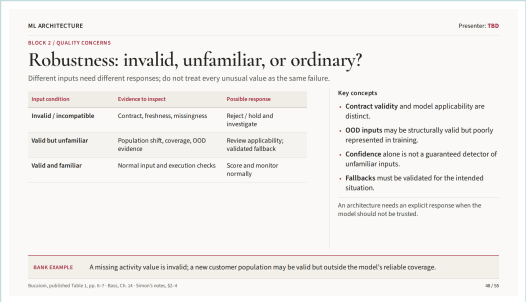
**SPOKEN EXAMPLE — practice this out loud**

*Choosing an acceptable model means weighing several different kinds of quality against each other, not just picking the most accurate one. Accuracy alone is a weak objective, especially when churners are a small minority of customers. Performance has to include the real end-to-end deadline and the resources you're spending. Complexity needs to earn its cost, both in training and in ongoing operation. What counts as acceptable genuinely depends on the task and on what an error actually costs. A heavier, fancier churn model is only worth it if its quality gain actually justifies the cost — and it still has to hit the campaign's deadline. Decide what 'good enough' means before you go looking for a fancier model.*

Reading anchor: Bucaioni, published Table 1, pp. 6–7 · Bass, Ch. 14 · Kalinowski, §13.4

## 48 / 55 Robustness: invalid, unfamiliar, or ordinary?

Block 2 / Quality concerns • TBD




**CUES**

- Presenter: **TBD**.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

**TO SAY**

- Different inputs need different responses; do not treat every unusual value as the same failure.
- **Contract validity** and model applicability are distinct.
- **OOD inputs** may be structurally valid but poorly represented in training.
- **Confidence** alone is not a guaranteed detector of unfamiliar inputs.
- **Fallbacks** must be validated for the intended situation.
- *At our bank,* a missing activity value is invalid; a new customer population may be valid but outside the model's reliable coverage.
- *An architecture needs an explicit response when the model should not be trusted.*

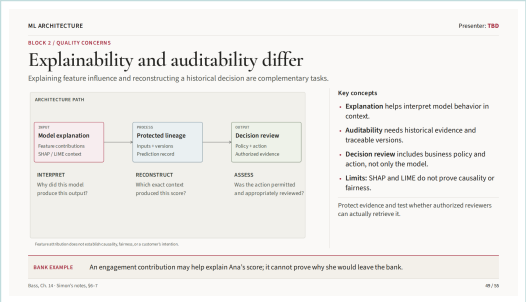
**SPOKEN EXAMPLE — practice this out loud**

*Not every unusual input deserves the same response, and it's worth telling three cases apart. Contract validity and whether the model actually applies well to this case are two separate questions. An out-of-distribution input might be perfectly valid by the contract, just poorly represented in what the model was trained on. And high confidence from the model is not, on its own, a reliable sign that an input is actually familiar to it. A missing activity value is simply invalid — that's a contract failure. A genuinely new customer population might be entirely valid data, yet still sit outside where the model is reliable. Your architecture needs an explicit, deliberate response for the moment the model probably shouldn't be trusted.*

Reading anchor: Bucaioni, published Table 1, pp. 6–7 · Bass, Ch. 14 · Simon's notes, §2–4

## 49 / 55 Explainability and auditability differ

Block 2 / Quality concerns • TBD




### CUES

- Presenter: **TBD**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Explaining feature influence and reconstructing a historical decision are complementary tasks.
- **Explanation** helps interpret model behavior in context.
- **Auditability** needs historical evidence and traceable versions.
- **Decision review** includes business policy and action, not only the model.
- **Limits**: SHAP and LIME do not prove causality or fairness.
- *At our bank,* an engagement contribution may help explain Ana's score; it cannot prove why she would leave the bank.
- *Protect evidence and test whether authorized reviewers can actually retrieve it.*

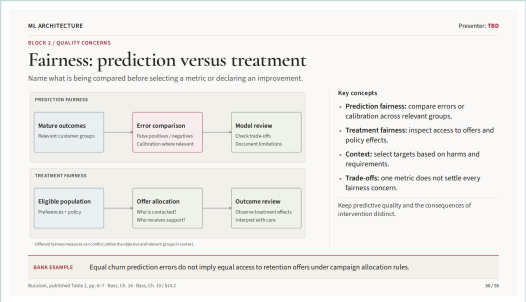
### SPOKEN EXAMPLE — practice this out loud

*Explaining why a model produced a particular output, and reconstructing exactly how a historical decision was made, are related but genuinely different jobs. An explanation helps you interpret the model's behavior in context. Auditability needs real historical evidence and versions you can actually trace. And reviewing a decision properly includes the business policy and the action taken — not just what the model output. It's worth saying plainly: tools like SHAP or LIME do not prove causality, and they do not prove fairness. An engagement feature's contribution might help explain why Ana's score came out the way it did — it cannot tell you why she would actually choose to leave the bank. The real test of auditability is whether an authorized reviewer can actually retrieve the evidence when it matters, not whether you drew a nice diagram of the process.*

Reading anchor: Bass, Ch. 14 · Simon's notes, §6–7

## 50 / 55 Fairness: prediction versus treatment

Block 2 / Quality concerns • TBD




### CUES

- Presenter: **TBD**.
- Animated build: click 3 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Name what is being compared before selecting a metric or declaring an improvement.
- **Prediction fairness**: compare errors or calibration across relevant groups.
- **Treatment fairness**: inspect access to offers and policy effects.
- **Context**: select targets based on harms and requirements.
- **Trade-offs**: one metric does not settle every fairness concern.
- *At our bank,* equal churn prediction errors do not imply equal access to retention offers under campaign allocation rules.
- *Keep predictive quality and the consequences of intervention distinct.*

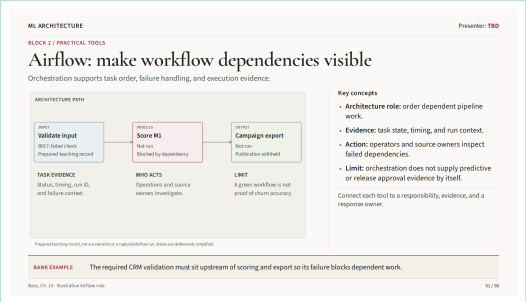
### SPOKEN EXAMPLE — practice this out loud

*Before picking a fairness metric, be precise about what you're actually comparing. Prediction fairness looks at whether errors or calibration differ across relevant groups. Treatment fairness looks at something different — who actually gets access to an offer, and what the policy's real effects are. Which target matters depends on context — on the actual harms at stake and what's required of you. And no single metric settles every fairness question on its own; there are real trade-offs between them. Equal error rates in churn prediction do not automatically mean equal access to retention offers once the campaign's own allocation rules are applied. Keep predictive quality and the real-world consequences of intervening cleanly separate in how you think about this.*

Reading anchor: Bucaioni, published Table 1, pp. 6–7 · Bass, Ch. 14 · Bass, Ch. 10 / §14.2

## 51 / 55 Airflow: make workflow dependencies visible

Block 2 / Practical tools • TBD




**CUES**

- New block: **Block 2 / Practical tools**.
- Presenter: **TBD**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

**TO SAY**

- Orchestration supports task order, failure handling, and execution evidence.
- **Architecture role**: order dependent pipeline work.
- **Evidence**: task state, timing, and run context.
- **Action**: operators and source owners inspect failed dependencies.
- **Limit**: orchestration does not supply predictive or release approval evidence by itself.
- *At our bank,* the required CRM validation must sit upstream of scoring and export so its failure blocks dependent work.
- *Connect each tool to a responsibility, evidence, and a response owner.*

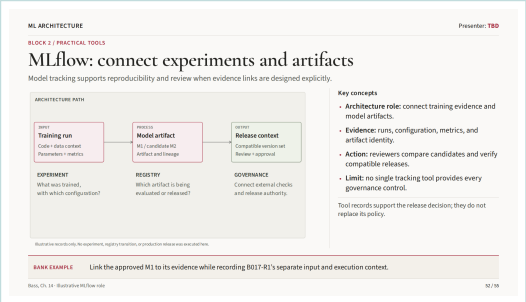
**SPOKEN EXAMPLE — practice this out loud**

*A tool like Airflow earns its place by making task order, failure handling, and execution evidence visible — it orders the dependent pieces of your pipeline. The evidence it gives you is task state, timing, and the context each run executed in. When something fails, operators and source owners can actually see and inspect that failed dependency. For us, the required CRM validation step has to sit upstream of scoring and export, specifically so that its failure blocks everything downstream from running on bad data. One honest limit, though: orchestration on its own gives you none of the evidence you need to judge predictive quality or approve a release — that's a different job entirely.*

Reading anchor: Bass, Ch. 14 · Illustrative Airflow role

## 52 / 55 MLflow: connect experiments and artifacts

Block 2 / Practical tools • TBD




**CUES**

- Presenter: **TBD**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

**TO SAY**

- Model tracking supports reproducibility and review when evidence links are designed explicitly.
- **Architecture role**: connect training evidence and model artifacts.
- **Evidence**: runs, configuration, metrics, and artifact identity.
- **Action**: reviewers compare candidates and verify compatible releases.
- **Limit**: no single tracking tool provides every governance control.
- *At our bank,* link the approved M1 to its evidence while recording B017-R1's separate input and execution context.
- *Tool records support the release decision; they do not replace its policy.*

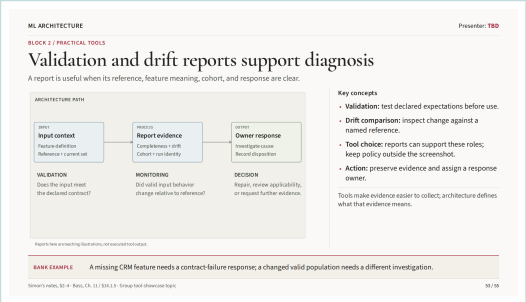
**SPOKEN EXAMPLE — practice this out loud**

*A tracking tool like MLflow supports reproducibility and review — but only if you actually design the evidence links deliberately. It connects your training evidence to your model artifacts. What it gives you is the run, the configuration, the metrics, and the artifact's identity. Reviewers use that to compare candidates and verify what's actually compatible before releasing. In our story, the approved M1 stays linked to its own evidence, while B017-R1 gets recorded separately, with its own distinct input and execution context. No single tracking tool hands you every governance control you need — it supports the release decision, it doesn't replace the policy behind it.*

Reading anchor: Bass, Ch. 14 · Illustrative MLflow role

## 53 / 55 Validation and drift reports support diagnosis

Block 2 / Practical tools • TBD




### CUES

- Presenter: **TBD**.
- Animated build: click 2 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- A report is useful when its reference, feature meaning, cohort, and response are clear.
- **Validation**: test declared expectations before use.
- **Drift comparison**: inspect change against a named reference.
- **Tool choice**: reports can support these roles; keep policy outside the screenshot.
- **Action**: preserve evidence and assign a response owner.
- *At our bank,* a missing CRM feature needs a contract-failure response; a changed valid population needs a different investigation.
- *Tools make evidence easier to collect; architecture defines what that evidence means.*

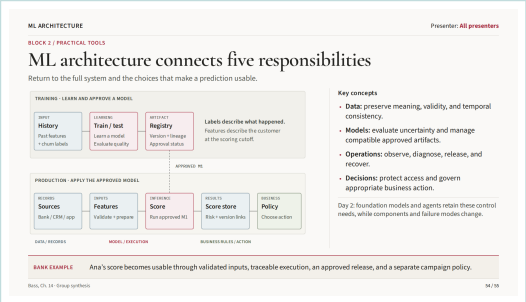
### SPOKEN EXAMPLE — practice this out loud

*A validation or drift report earns its usefulness from four things being clear: its reference, what the feature actually means, which cohort it covers, and who's supposed to respond to it. Validation tests the expectations you've actually declared, before anything gets used downstream. Drift comparison checks change against a reference you've explicitly named. Tools can genuinely help with both of these roles — just keep the actual policy decisions out of the screenshot itself. And always, preserve the evidence and assign someone to respond to it. A missing CRM feature needs a contract-failure response; a population that's shifted but still valid needs a completely different kind of investigation. Tools make evidence easier to gather — the architecture is what decides what that evidence actually means.*

Reading anchor: Simon's notes, §2–4 · Bass, Ch. 11 / §14.1.5 · Group tool-showcase topic

## 54 / 55 ML architecture connects five responsibilities

Block 2 / Practical tools • All presenters




### CUES

- Presenter: **All presenters**.
- Animated build: click 4 times to reveal the remaining steps before moving on.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- Return to the full system and the choices that make a prediction usable.
- **Data**: preserve meaning, validity, and temporal consistency.
- **Models**: evaluate uncertainty and manage compatible approved artifacts.
- **Operations**: observe, diagnose, release, and recover.
- **Decisions**: protect access and govern appropriate business action.
- Ana's score becomes usable through validated inputs, traceable execution, an approved release, and a separate campaign policy.
- *Day 2: foundation models and agents retain these control needs, while components and failure modes change.*

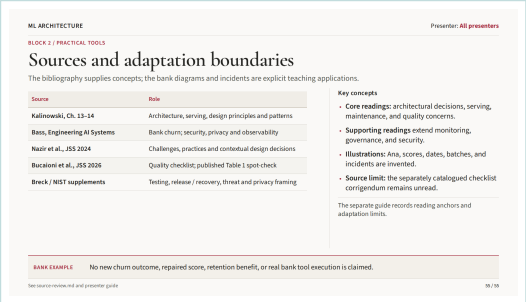
### SPOKEN EXAMPLE — practice this out loud

*Let's step back and look at the whole system one more time, and what actually makes a prediction usable. Data has to preserve its meaning, its validity, and consistency over time. Models need their uncertainty evaluated and their approved, compatible versions carefully managed. Operations means observing, diagnosing, releasing, and recovering — all as an ongoing discipline, not a one-time setup. And decisions means protecting access and properly governing whatever business action follows. Trace it back to Ana: her score only becomes genuinely usable through validated inputs, execution you can actually trace, a model release that went through approval, and a campaign policy that's kept entirely separate from the prediction itself. Tomorrow, on Day 2, we'll see that foundation models and agents keep needing exactly these same controls — even as the components and the ways things fail change underneath them.*

Reading anchor: Bass, Ch. 14 · Group synthesis

## 55 / 55 Sources and adaptation boundaries

Block 2 / Practical tools • All presenters




### CUES

- Presenter: **All presenters**.
- Pause briefly before the bank example — let the concept land first.
- Close on the takeaway line before advancing.

### TO SAY

- The bibliography supplies concepts; the bank diagrams and incidents are explicit teaching applications.
- **Core readings**: architectural decisions, serving, maintenance, and quality concerns.
- **Supporting readings** extend monitoring, governance, and security.
- **Illustrations**: Ana, scores, dates, batches, and incidents are invented.
- **Source limit**: the separately catalogued checklist corrigendum remains unread.
- *At our bank,* no new churn outcome, repaired score, retention benefit, or real bank tool execution is claimed.
- *The separate guide records reading anchors and adaptation limits.*

### SPOKEN EXAMPLE — practice this out loud

*One last thing before we close. Our bibliography is where the real concepts come from; the bank diagrams and the incidents you've seen today are our own invented teaching applications of those concepts, built to make them concrete. The core readings cover architectural decisions, serving, maintenance, and quality; the supporting readings extend into monitoring, governance, and security. To be completely clear: Ana, her scores, the dates, the batches, the incident — all of it is invented for this lecture. No real churn outcome, no real repaired score, no real retention benefit, and no real bank tool execution is being claimed anywhere in this talk. A separate guide records exactly where each idea in this lecture actually comes from, and the limits of how we've adapted it.*

Reading anchor: See source-review.md and presenter guide